

CZĘŚĆ 1

Wstęp - Przeanalizuj i wykonaj skrypty. Udokumentuj wszystko zrzutami ekranu.

Linux: Wiersz poleceń – skrypty bash – wprowadzenie

Wprowadzenia słów kilka do skryptów powłoki. Zmienne, wypisywanie na ekran, wprowadzanie z klawiatury, parametry, funkcje, instrukcja warunkowa if.

Zwyczajowo pliki skryptów bash oznaczamy "rozszerzeniem" .sh, natomiast jako pierwszą linię wprowadzamy #!/bin/bash.

rozpoczyna komentarz – informacyjny fragment linii, bez znaczenia dla działania programu.

Czas na pierwszy skrypt:

```
1. student@student:~/Pulpit$ cat > skrypt1.sh
2. #!/bin/bash
3. echo "No cześć"
```

Skrypt już mamy. Po jego niezwykle skomplikowanej treści możemy się domyśleć co zrobi, jednak najpierw ustawmy jeszcze dla wszystkich użytkowników prawo jego wykonywania.

```
1. student@student:~/Pulpit$ ls -l
2. -rw-rw-r-- 1 student student 30 mar 20 16:41 skrypt1.sh
3.
4. student@student:~/Pulpit$ chmod +x skrypt1.sh
5.
6. student@student:~/Pulpit$ ls -l
7. -rwxrwxr-x 1 student student 30 mar 20 16:41 skrypt1.sh
```

Czas zobaczyć nasze dzieło w działaniu:

```
1. student@student:~/Pulpit$ bash skrypt1.sh
2. No cześć
```

read – wczytanie informacji ze standardowego wejścia, klawiatury od użytkownika

Wykorzystajmy tą wiedzę i napiszmy skrypt, który zapyta użytkownika o imię, następnie wypisze je na ekran:

```
1. #!/bin/bash
2. echo "Podaj swoje imię:"
3. read imię
4. echo "Podane imię to:$imię"
```

Efekt działania:

```
1. student@student:~/Pulpit$ bash imię.sh
2. Podaj swoje imię:
3. TechnikInformatyk.pl
4. Podane imię to:TechnikInformatyk.pl
```

Każde wykonywana funkcja zwraca kod, kod sukcesu bądź błędu. W przypadku pozytywnego wykonania kodem standardowo jest 0.

Sprawdźmy jakie kody zostaną zwrócone w poniższym przykładzie:

```
1. student@student:~/Pulpit$ ls /etc/passwd
2. /etc/passwd
3. student@student:~/Pulpit$ echo $?
4. 0
5. student@student:~/Pulpit$ ls /etc/passwdabc
6. ls: nie ma dostępu do '/etc/passwdabc': Nie ma takiego pliku ani katalogu
7. student@student:~/Pulpit$ echo $?
8. 2
```

Umieszczenie wielu poleceń w jednej linii. Przeanalizujmy jakie różnice nastąpią przy łączeniu poleceń za pomocą symboli : && oraz ||.

```
1. student@student:~/Pulpit$ dir
2. plik2 plik3
3.
4. student@student:~/Pulpit$ cat plik1 ; cat plik2 ; cat plik3
5. cat: plik1: Nie ma takiego pliku ani katalogu
6. zawartość pliku 2
7. zawartość pliku 3
8.
9. student@student:~/Pulpit$ cat plik1 && cat plik2 && cat plik3
10. cat: plik1: Nie ma takiego pliku ani katalogu
11.
12. student@student:~/Pulpit$ cat plik1 || cat plik2 || cat plik3
13. cat: plik1: Nie ma takiego pliku ani katalogu
14. zawartość pliku 2
```

W przypadku "i", logicznej koniunkcji, czyli symbolu podwójnego & jeśli dane polecenie będzie wykonane z błędem, kolejne nie będą wykonywane.

Logiczna alternatywa, || słownie lub, przechodzi przez kolejne polecenia, aż wykonanie któregoś nie zakończy się kodem 0.

Symbol ; jedynie rozdziela kolejne polecenia. Wszystkie one zostaną wykonane.

Wewnątrz skryptu możemy odwołać się do parametrów. Parametrem zerowym jest nazwa skryptu, kolejne możemy podać podczas uruchamiania skryptu.

Parametr	Znaczenie
\$0	Nazwa skryptu
\$1	Pierwszy parametr
\$2, \$3, etc.	Drugi parametr, trzeci parametr, ...
\$*	Wszystkie parametry
\$#	Liczba argumentów

Przykładowa zawartość skryptu odczytującego parametry wejściowe:

```
1. #!/bin/bash
2. echo "Nazwa skryptu: "$0
3. echo "Pierwszy parametr: "$1
4. echo "Drugi parametr: "$2
5. echo "Wszystkie parametry: "$*
```

Oraz efekt jego działania:

```
1. student@student:~/Pulpit$ bash parametry.sh 20 TI
2. Nazwa skryptu: parametry.sh
3. Pierwszy parametr: 20
4. Drugi parametr: TI
5. Wszystkie parametry: 20 TI
```

Jeśli dany fragment kodu skryptu planujemy wykonywać wielokrotnie, warto zamknąć go w strukturę funkcji – elementu do którego odwołujemy się podając jej nazwę.

Przykład zastosowania funkcji:

```
1. #!/bin/bash
2.
3. ulubiony(){
4.     echo Mój ulubiony przedmiot to: $1
5. }
6. echo ""
7. ulubiony "Język Polski"
8. ulubiony Matematyka
9. ulubiony WF
10. ulubiony Biologia
```

Efekt działania:

```
1. student@student:~/Pulpit$ bash funkcje.sh
2.
3. Mój ulubiony przedmiot to: Język Polski
4. Mój ulubiony przedmiot to: Matematyka
5. Mój ulubiony przedmiot to: WF
6. Mój ulubiony przedmiot to: Biologia
```

Ostatnim elementem tego przydługiego tematu będzie instrukcja warunkowa. Pytanie jeżeli, zapytanie o spełnienie warunku oraz wykonanie pewnych czynności w przypadku gdy ten warunek będzie spełniony oraz innych gdy spełniony nie będzie.

Wersja kompaktowa:

```
1. if WARUNKI; then INSTRUKCJE; fi
```

Wersja standardowa:

```
1. if condition
2. then
3.     statements
4. else
5.     statements
6. fi
```

Przykładowy skrypt, który sprawdza czy istnieje ścieżka /etc/passwd.

```
1. #!/bin/bash
2.
3. if [ -f /etc/passwd ]
4. then
5.     echo "/etc/passwd istnieje."
6. fi
```

Efekt działania:

```
1. student@student:~/Pulpit$ bash jezeli.sh
2. /etc/passwd istnieje.
```

Skrypt przeanalizował strukturę katalogową systemu Linux i stwierdził, że ścieżka /etc/passwd istnieje.

W powyższym programie wykorzystaliśmy parametr `-f` sprawdzający czy plik istnieje i jest plikiem 😊
Poniżej tabela wszystkich możliwych opcji dotyczących pliku.

Parametr	Znaczenie
<code>-e plik</code>	Sprawdzenie czy plik istnieje.
<code>-d plik</code>	Sprawdzenie czy plik jest katalogiem.
<code>-f plik</code>	Sprawdzenie czy plik jest plikiem (a nie np. dowiązaniem symbolicznym).
<code>-s plik</code>	Sprawdzenie czy rozmiar pliku jest większy od 0.
<code>-u plik</code>	Sprawdzenie czy plik ma ustawioną flagę suid – w przypadku SUID ustawionego na pliku wykonywalnym powoduje wykonanie pliku z UID ustawionym na właściciela pliku (jeżeli właścicielem pliku jest root, to uruchomienie takiego pliku z dowolnego konta spowoduje wykonanie tego pliku z poziomem uprawnień roota).
<code>-g plik</code>	Sprawdzenie czy plik ma ustawioną flagę sgid .
<code>-r plik</code>	Sprawdzenie czy plik jest odczytywalny.
<code>-w plik</code>	Sprawdzenie czy plik jest zapisywalny.
<code>-x plik</code>	Sprawdzenie czy plik jest wykonywalny.

Napiszmy zatem kolejny skrypt. Tym razem sprawdzamy czy plik o nazwie `plik1` istnieje. Jeśli istnieje wypisujemy taką właśnie informację na ekran, jeśli nie istnieje – skrypt go utworzy.

```
1. #!/bin/bash
2.
3. if [ -f plik1 ]
4. then
5.     echo Plik1 już istnieje
6. else
7.     echo Utworzony > plik1
8.     echo Plik1 został utworzony
9.
10. fi
```

Efekt dwukrotnego uruchomienia skryptu w katalogu, gdzie plik `plik1` nie istniał

```
1. student@student:~/Pulpit$ bash jeze1f2.sh
2. Plik1 został utworzony
3.
4. student@student:~/Pulpit$ bash jeze1f2.sh
5. Plik1 już istnieje
```

Sprawdzaliśmy za pomocą instrukcji warunkowej jeżeli czy plik istniał, zabierzmy się na chwilę za sprawdzenie wartości liczbowych.

Możliwe operacje na liczbach:

Operator	Znaczenie
-eq	Równe
-ne	Nierówne
-gt	Większe niż
-lt	Mniejsze niż
-ge	Większe lub równe
-le	Mniejsze lub równe

Przykładowy skrypt

```
1. #!/bin/bash
2.
3. WIEK=$1
4.
5. if [[ $WIEK -ge 0 ]] && [[ $WIEK -lt 11 ]]; then
6.     echo "Jesteś dzieckiem..."
7. elif [[ $WIEK -ge 11 ]] && [[ $WIEK -lt 20 ]]; then
8.     echo "Jesteś nastolatkiem..."
9. elif [[ $WIEK -ge 20 ]] && [[ $WIEK -lt 30 ]]; then
10.    echo "Jesteś dzieścią..."
11. elif [[ $WIEK -ge 30 ]] && [[ $WIEK -lt 50 ]]; then
12.    echo "Jesteś dzieścią..."
13. elif [[ $WIEK -ge 50 ]] && [[ $WIEK -lt 100 ]]; then
14.    echo "Jesteś dziesiąt..."
15. elif [[ $WIEK -ge 100 ]] && [[ $WIEK -lt 130 ]]; then
16.    echo "Gratulacje..."
17. else echo "Podano nieprawidłowe dane wejściowe"
18. fi
```

oraz efekt jego zastosowania dla różnych wartości liczbowych podanych jako parametr wejściowy nr 1.

```
1. student@student:~/Pulpit$ bash wiek.sh 5
2. Jesteś dzieckiem...
3. student@student:~/Pulpit$ bash wiek.sh 15
4. Jesteś nastolatkiem...
5. student@student:~/Pulpit$ bash wiek.sh 35
6. Jesteś dzieścią...
7. student@student:~/Pulpit$ bash wiek.sh 75
8. Jesteś dziesiąt...
9. student@student:~/Pulpit$ bash wiek.sh 105
10. Gratulacje...
```